Project: Designing a Telemetry and UX Database for *Chocolate-Doom* Research



DBS – Semester Project Brief

October 20, 2026

Contents

1 Context and Motivation 1

2 Learning Objectives 2

3 Data Model (Conceptual →Logical) 2

4 Data Ingestion (ETL) Guidance 3

5 Analytics Queries 3

6 Project Tasks & Deliverables 4

7 Submission Format 4

8 Grading Rubric (100 pts) 5

9 Bonus 5

1 Context and Motivation

A research group is collecting gameplay telemetry from a *hacked* version[1] of `chocolate-doom`. This build emits per-tic data (on screen and/or to file) that includes the player position (x, y, z), facing *angle*, *momentum* vector, *point-of-view* (FOV/camera), and combat stats (health, armor, ammo). It also logs meta-information such as *tic* number, *episode*, *map*, and *sector*.

The team aims to aggregate data across multiple play sessions to identify movement trends and behavioral patterns. This analysis will facilitate the detection of player cooperation or potential sabotage. Student volunteers provide demographics (age, gender, experience) and complete *one* of these UX instruments:

[1] The source code is available at `https://github.com/aocalderon/trajectory_doom`

- **PENS** (Player Experience of Need Satisfaction).
- **GUESS** (Game User-Experience Satisfaction Scale).
- **BANGS** (Basic Needs in Games; open-access).

Goal: Design and prototype a relational database that ingests telemetry and survey data, supports exploratory queries and analytics for movement/cooperation trends, and enforces data quality and research ethics.

2 Learning Objectives

By completing this project, you will:

1. Model a real-world domain into entities, attributes, and relationships (ER & relational schema).
2. Normalize tables to at least 3NF (justify any denormalizations for performance).
3. Define keys, constraints, and reference integrity for high-frequency telemetry.
4. Design an ingestion pipeline for semi-structured logs (TSV $\rightarrow$ staging $\rightarrow$ core).
5. Implement indices and assess their impact with query plans and timings.
6. Formulate SQL queries for trajectory, proximity, and cooperation analyses.
7. Integrate user demographics and UX scales (PENS/GUESS/BANGS) with telemetry.
8. Address privacy, consent, and research-ethics constraints in schema & process.

3 Data Model (Conceptual $\rightarrow$Logical)

Movement & Cooperation Signals (to inform schema/queries)

The research group is interested on find useful patter as follows:

- *Trajectories*: sequence of (x, y) ordered by tic per player per game.
- *Proximity events*: players within a spatial threshold for $\geq k$ seconds.
- *Co-occurrence in sectors*: overlapping time in same sector (optional: adjacent sectors).

Conceptual ER (deliverable)

Produce an ER diagram capturing main relationships. Identify key entities and relationships; you are encouraged to propose your own.

Indexing

You must implement and evaluate a number of valid index over your proposed tables.

4 Data Ingestion (ETL) Guidance

Assume the hacked engine emits TSV lines.
Recommended pipeline:

1. Load raw logs to a *staging* table (text fields) with minimal constraints.
2. Validate & transform into typed core tables using `INSERT ... SELECT`.
3. Deduplicate on `(game_id, tic, player_id)`; reject malformed records with an *error log* table.

5 Analytics Queries

Implement SQL solutions for **any six (6) of the eight (8)** proposed analytical queries. *Note: The provided hints utilize generic table and attribute names; you are required to adapt these to the specific nomenclature defined in your database schema.*

1. Average duration of game sessions per map.
   Hint: Use the 'Game' table and the `AVG` aggregation function on the 'timestamp' or 'tic' field, grouped by the 'map_id'.
2. Players with the highest average proximity.
   Hint Perform a self-join on the 'TelemetryEvent' table (matching on 'game_id' and/or 'tic' – actually I am not sure) to calculate the *Euclidean distance*[2] between player pairs, then filter for proximity thresholds (maybe <= 5.0?) and aggregate the results to find the pairs with the highest average closeness.
3. Shortest and longest trajectory distances per player.
   Hint Calculate the total length of each game's trajectory by summing the Euclidean distances between consecutive tics (using a self-join or *window function*[3]), then group by player to find the minimum and maximum of those totals.
4. List UX survey responses for players with above-average trajectories duration.
   Hint Calculate the average duration of all trajectories (using total tics or time differences), then use a subquery to select players whose individual duration exceeds this global average and JOIN them to the 'UXResponse' table to retrieve their survey answers.
5. Most Visited Sector (Hotspot) per Episode and Map.
   Hint Group the 'TelemetryEvent' records by 'episode', 'map', and 'sector'. The sector should be a cell in a grid of a 250×250 units covering the current map. Then use the `COUNT` function to calculate the frequency of player presence in each area, sorting the results in descending order to identify the hotspots.
6. Number of Tics Where Players Were Together in a Sector.
   Hint Perform a self-join on the 'TelemetryEvent' table matching records by 'game_id', 'tic', and 'sector_id' to identify co-presence while filtering for distinct 'player_ids', and then apply `COUNT(DISTINCT tic)` to calculate the total duration of these overlapping moments.
7. Average UX Score for Players with the Shortest Trajectory per Episode.
   Hint Identify the players with the minimum total trajectory distance for each episode (using a *window function* or subquery), then JOIN those players to the 'UXResponse' table to calculate the average of their survey scores.

[2] https://en.wikipedia.org/wiki/Euclidean_distance
[3] If you use window functions, that will be a bonus.

8. Total Distance Traveled and Average Speed per Player, Analyzing All Games for a Player.
   Hint Sum the Euclidean distances between consecutive tics across all sessions to determine the total distance, then divide this by the total duration (calculated from the aggregated tic counts or game timestamps) to derive the average speed, grouping the results by player identity.

6 Project Tasks & Deliverables

Part A: Conceptual and Logical Design (Week 1)

1. Write assumptions and requirements (functional/non-functional, ethics).
2. Produce an initial draft of ER diagram with cardinalities and key attributes.
3. Derive relational schema; list all FKs, PKs, and constraints (Data Dictionary - DD) from the draft.

Part B: Implementation & Ingestion (Week 2–3)

1. Complete ER, Schema and DD.
2. Implement DDL in your DBMS of preference (PostgreSQL recommended).
3. Create scripts and *optional* staging tables to load sample telemetry logs (TSV).
4. Populate the *UX Instrument* tables with the survey of your choice (PENS,GUESS or BANGS) using real data for each member of the team.
5. Insert real data (at least 3 different episodes, 6+ players, $\geq$ 20k telemetry rows) (yes, real data means you have to play Doom!).

Part C: Queries, Indexing, and Reporting (Week 4–5)

1. Implement at least 6 analytical queries (see Section 5).
2. Create at least 3 indexes. Show `EXPLAIN(ANALYZE)` before/after (using tables and plots of average runs). Discuss any particular finding.
3. Provide 2 views and 1 materialized view for frequent analyses.
4. Provide and document a simple mechanism (could be a `Makefile` or shell script) to recreate the schema and populate your data (via dumps or backups).

7 Submission Format

Submit a single **PDF** report with:

- **ER diagram**, **relational schema**, and **rationale** (brief explanation about your model and schema).
- **Data dictionary**.
- **DDL/constraints** (appendix with code snippets).
- **ETL description** + sample of raw telemetry.
- **Queries + results** (screenshots/tables).
- **Index evaluation**.
- **Views implementation**.

8 Grading Rubric (100 pts)

Criterion	Points
Problem framing, assumptions, requirements clearly stated	10
Conceptual ER correctness (entities, keys, cardinalities) & data dictionary	25
Relational schema & constraints	10
Implementation quality (DDL, integrity, sample data)	10
ETL pipeline (staging → core, validation)	10
Analytics queries (6+) correctness & insight	15
Indexing & performance evaluation (EXPLAIN/ANALYZE)	10
Views/materialized view for reuse	5
Report quality (clarity, organization, reproducibility)	5
Total	**100**

Note: In the final exam there will be up to **2 open SQL questions** about the ER of this project.

9 Bonus

- If available, enable extensions like `citext`, `uuid-ossp` or `pgcrypto` for authorization or `postgis` for spatial indexing and analysis.

RIP & TEAR



